TiDB as an agentic coding shared memory system

Claude Code, Codex, and Cursor all reading and writing the same TiDB knowledge graph over MCP - so AI-written infrastructure stops duplicating work, breaking conventions, and starting from zero.

Claude Code | Codex | Cursor | Model Context Protocol | Recursive CTE graph

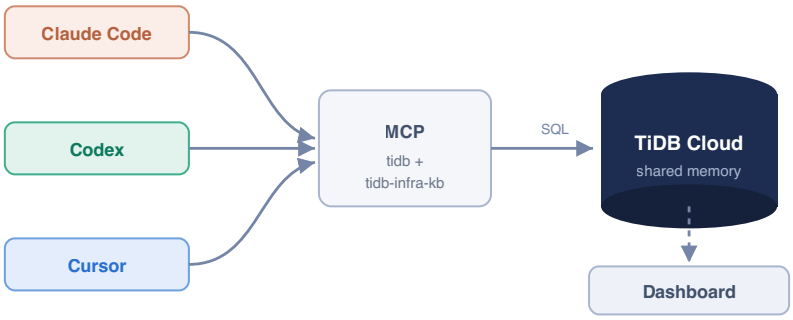
Technical demo report

Backed by a live TiDB Cloud cluster

What this is

Teams now write infrastructure with AI coding tools. But each tool works **blind and alone** - it cannot see what already exists, the team's conventions, or how components connect. The result is duplication, drift, broken conventions, and outages.

This demo puts the engineering knowledge where every tool can use it: a **knowledge graph in TiDB Cloud**. Every coding tool connects to it over the **Model Context Protocol (MCP)**, queries it before building, writes its work back, and traverses component relationships with **recursive SQL**. The example workload is an infrastructure-as-code monorepo (Pulumi / TypeScript) for a fictional company, `Acme`, spanning AWS, Cloudflare, and Okta.



Three real CLIs → two MCP servers → one TiDB database. The dashboard reads the same database live.

> **Proven on real TiDB - every screenshot below is from an actual run.** The graph, the writes from three real CLIs (Claude Code, Codex, Cursor), and the recursive-CTE traversals all ran against a live TiDB Cloud cluster (v8.5.3) over the MySQL protocol with TLS. The terminal captures are the unedited tool output.

The problem: AI tools with no shared memory

Four failure modes show up the moment more than one agent (or session) touches the same codebase.

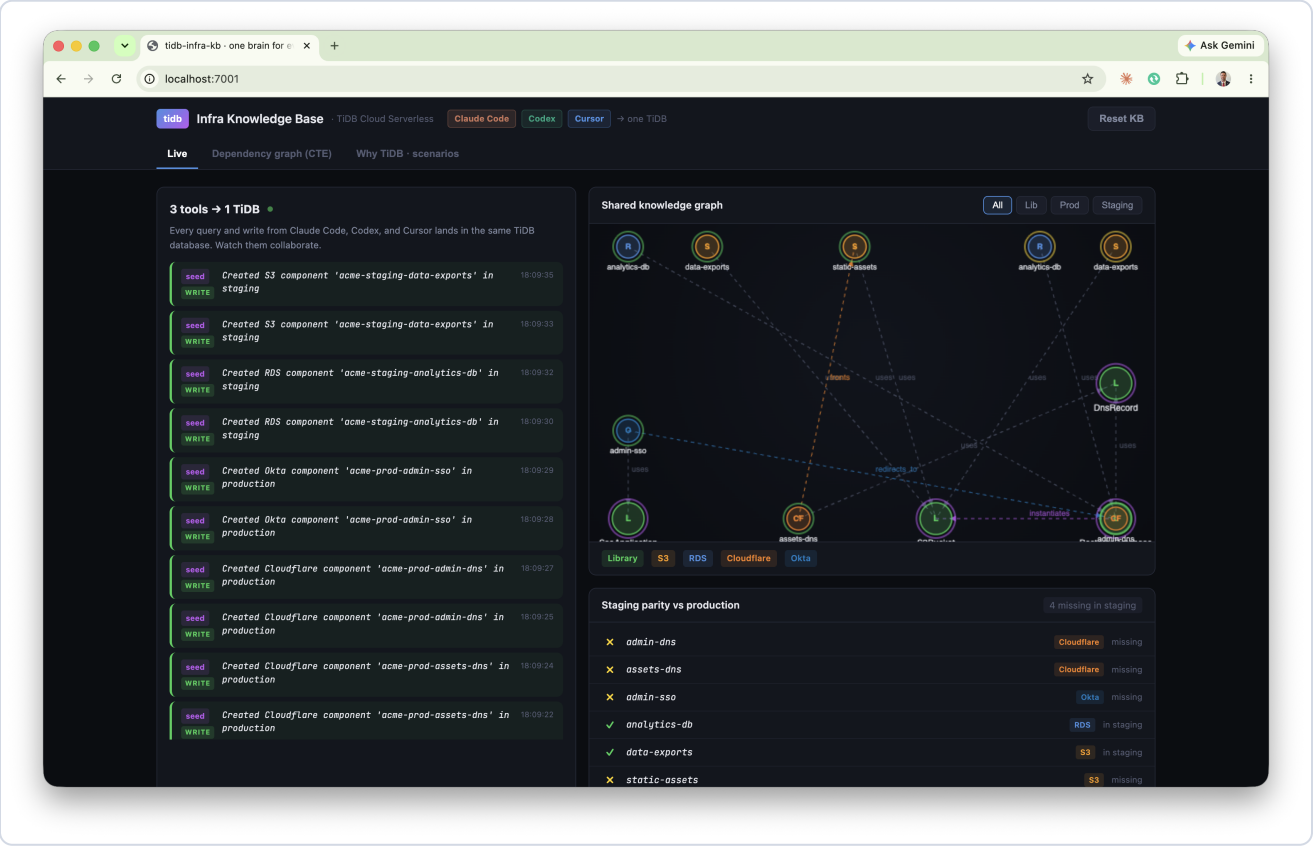
FAILURE MODE	WHAT HAPPENS WITHOUT A SHARED GRAPH
Duplication	A tool can't see a component already exists, so it builds a second one. Drift and double cost follow.
Convention breaks	It writes a raw `aws.s3.Bucket` instead of the approved library wrapper, skipping required tags and policies.
Invisible blast radius	It changes a shared library and breaks resources two hops away - discovered only in production.
Cold starts	The next tool or session re-derives everything from the filesystem and re-makes the same mistakes.

The common root cause: the knowledge that prevents these - **what exists, how it connects, and the rules** - lives only in scattered code and people's heads. It is not queryable, and it is not shared across tools.

> **The fix.** Model the codebase as a graph in TiDB: `infra_components` (nodes), `component_edges` (typed relationships), and `session_log` (who did what). Expose it to every tool over MCP. Now the tools query before they build and hand off warm.

The live demo: three tools, one task

The task: production is complete, but **staging is missing its DNS and SSO layer**. Watch three different AI coding tools finish it - each picking up where the last left off, because they share one TiDB brain.



Starting state. Left: the live activity feed (one row per query/write from any tool). Right: the knowledge graph and the staging-parity strip - four components are missing in staging (red ✗).

Step 1 - Claude Code inspects, then builds the asset layer claude-code

It first **queries** the graph (it does not assume), then scaffolds the missing static-assets bucket and its Cloudflare DNS record by composing the existing `S3Bucket` and `DnsRecord` libraries - never raw cloud resources - and **writes each one back**.

● Both components recorded (IDs 25 and 26). Here's a summary of everything done:

Gap found - staging missing vs production:

Resource	Status
acme-staging-static-assets (S3)	Scaffolded
acme-staging-assets-dns (Cloudflare)	Scaffolded
acme-staging-admin-dns (Cloudflare)	Out of scope
acme-staging-admin-sso (Okta)	Out of scope

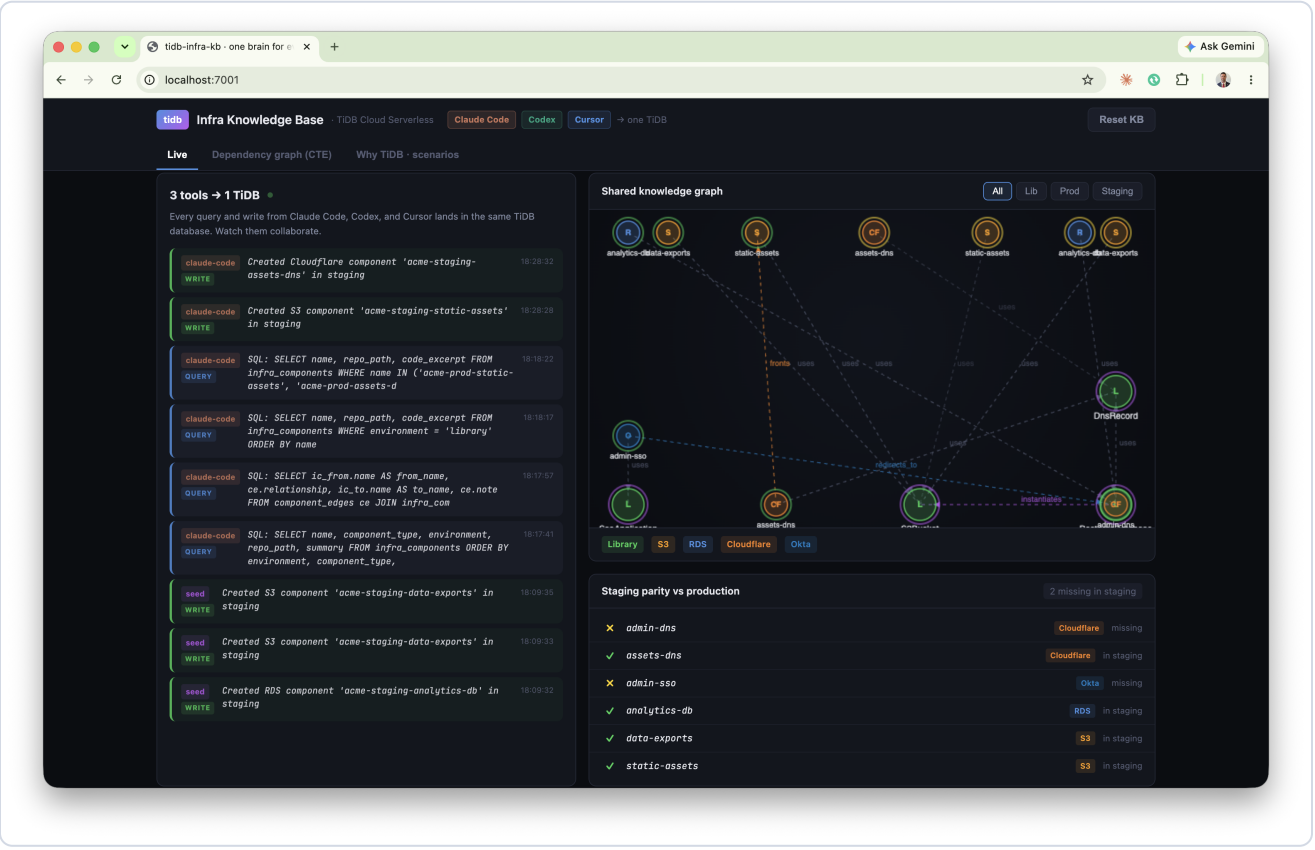
Files created:

- environments/staging/storage.ts - adds acme-staging-static-assets via S3Bucket (existing acme-staging-data-exports retained)
- environments/staging/dns.ts - adds acme-staging-assets-dns via DnsRecord, wired to staticAssets.bucketDomainName

Rules followed:
- No raw aws.* resources - both use the library wrappers (S3Bucket, DnsRecord)
- proxied: true is library-enforced on DnsRecord (never overridden)
- All four required tags present: Environment, ManagedBy, Component, Team
- Physical names follow acme-<env>-<name> convention
- Both components recorded in the KB with depends_on edges to their respective libraries

✻ Baked for 2m 14s

Real Claude Code output. It recorded both components (IDs 25 & 26) and reports the rules it followed: no raw `aws.*`, `proxied: true` library-enforced, all four required tags, `acme-<env>-<name>` naming, and KB writeback with dependency edges.



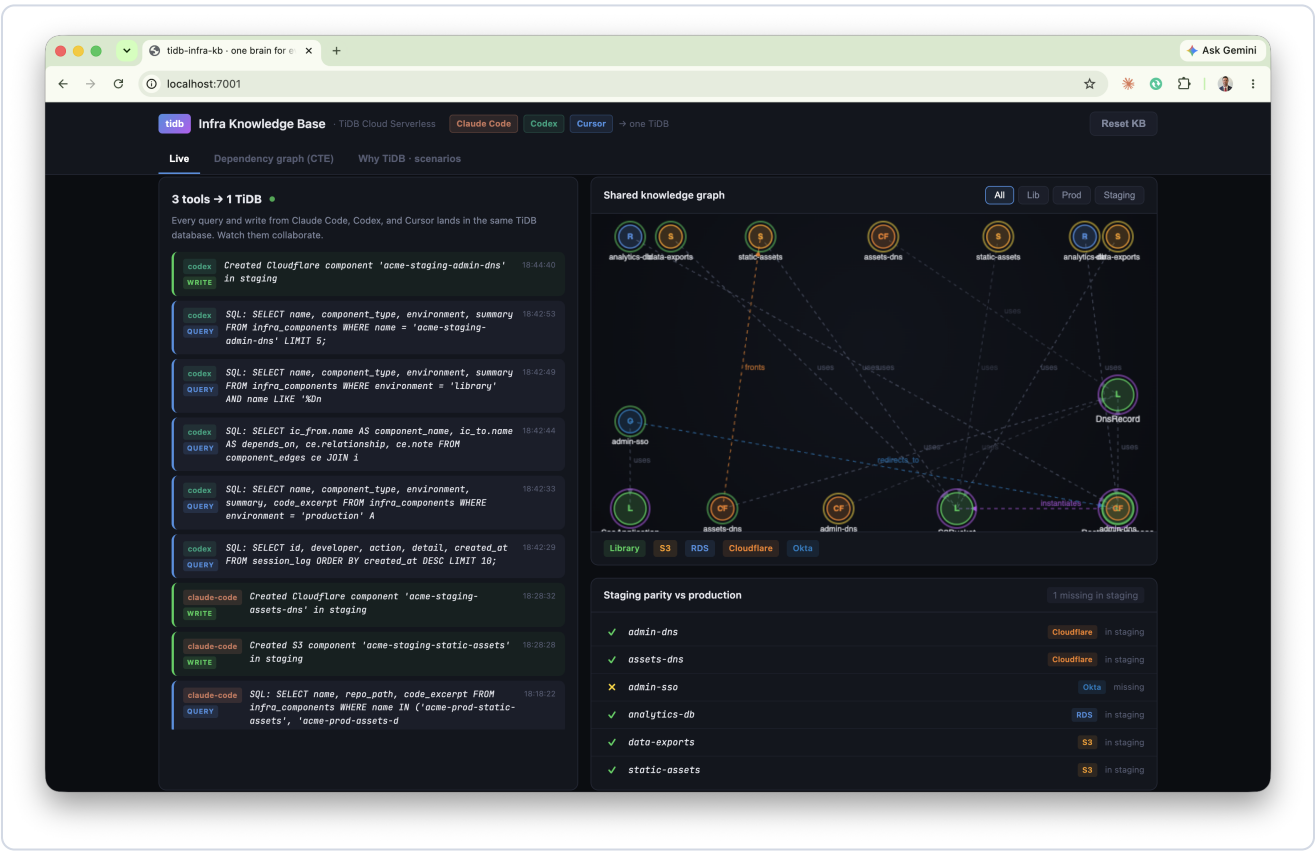
The dashboard after Claude Code: `claude-code` QUERY rows (it checked first) then two WRITE rows; the staging gap drops from 4 to 2.

Step 2 - Codex picks up warm `codex`

Codex opens with **zero local memory** of Claude Code's work. It reads the shared `session_log`, sees exactly what was just created, and continues with the admin-portal DNS record. No copy-pasting context between tools.

```
• I have enough context to continue the build consistently. I'm recording acme-staging-admin-dns now as a staging Cloudflare component that
  composes DnsRecord, with the summary mirrored from production and adjusted for staging.

• Called
  └ tidb-infra-kb.write_component({"depends_on":"DnsRecord","developer":"codex","env":"staging","name":"acme-staging-admin-
        dns","summary":"Cloudflare DNS record for the admin portal. acme-staging-admin-sso redirects here.","type":"Cloudflare"})
    Written: acme-staging-admin-dns (id=27)

• The previous session most recently created acme-staging-assets-dns, immediately after acme-staging-static-assets.

  I continued the staging build by recording acme-staging-admin-dns as a Cloudflare component in staging, depending on DnsRecord, with the
  production-matching summary adapted for staging. write_component succeeded with id 27.

─ Worked for 5m 18s ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

Real Codex output: "The previous session most recently created `acme-staging-assets-dns`" - it read the shared log, then wrote `acme-staging-admin-dns` (id=27). A different tool, continuing seamlessly.



The dashboard now shows `codex` rows above the `claude-code` rows - two tools, one feed, one database. Gap down to 1.

Step 3 - Cursor verifies with a recursive CTE, then finishes cursor

Before writing the SSO application, Cursor runs a **recursive query** over the graph to confirm the dependency pattern (an SSO app must redirect to a DNS record), then scaffolds it correctly. Staging reaches parity.

Recursive CTE: acme-prod-admin-sso transitive dependencies

Ran the project's downstream CTE via `db_query`:

Depth	From	Relationship	To	Type
1	acme-prod-admin-sso	uses	SsoApplication	Library
1	acme-prod-admin-sso	redirects_to	acme-prod-admin-dns	Cloudflare
2	acme-prod-admin-dns	uses	DnsRecord	Library

Transitive closure: **SsoApplication** → **acme-prod-admin-dns** → **DnsRecord**.

SSO must redirect to a DnsRecord

Confirmed in the graph and conventions:

- Production has acme-prod-admin-sso **redirects_to** acme-prod-admin-dns, and that DNS record **uses** DnsRecord.
- SsoApplication is documented as requiring redirect URIs on the matching DnsRecord hostname (libs/sso-app, configs/claude-code/CLAUDE.md).

So the SSO app does not redirect to a raw hostname in isolation — it redirects to the **admin DNS component**, which composes **DnsRecord**.

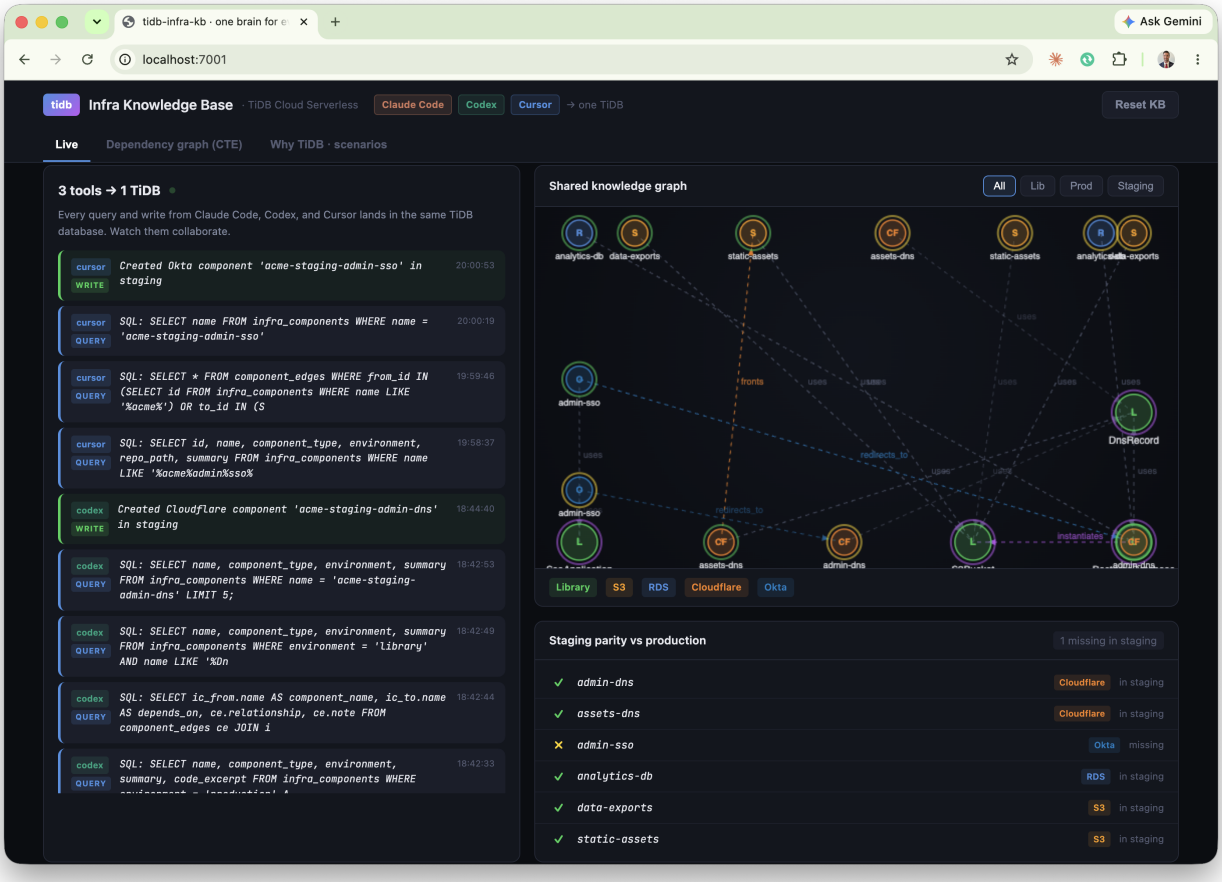
Staging scaffold + KB writeback

1. **environments/staging/dns.ts** — added acme-staging-admin-dns and adminDnsHostname (needed for the redirect URI).
2. **environments/staging/sso.ts** — new SsoApplication with redirect https://${adminDnsHostname}/callback.
3. **write_component** — recorded acme-staging-admin-sso (Okta, staging, composes SsoApplication).
4. **db_execute** — edge acme-staging-admin-sso **redirects_to** acme-staging-admin-dns.

KB edges for staging SSO:

- uses → SsoApplication
- redirects_to → acme-staging-admin-dns

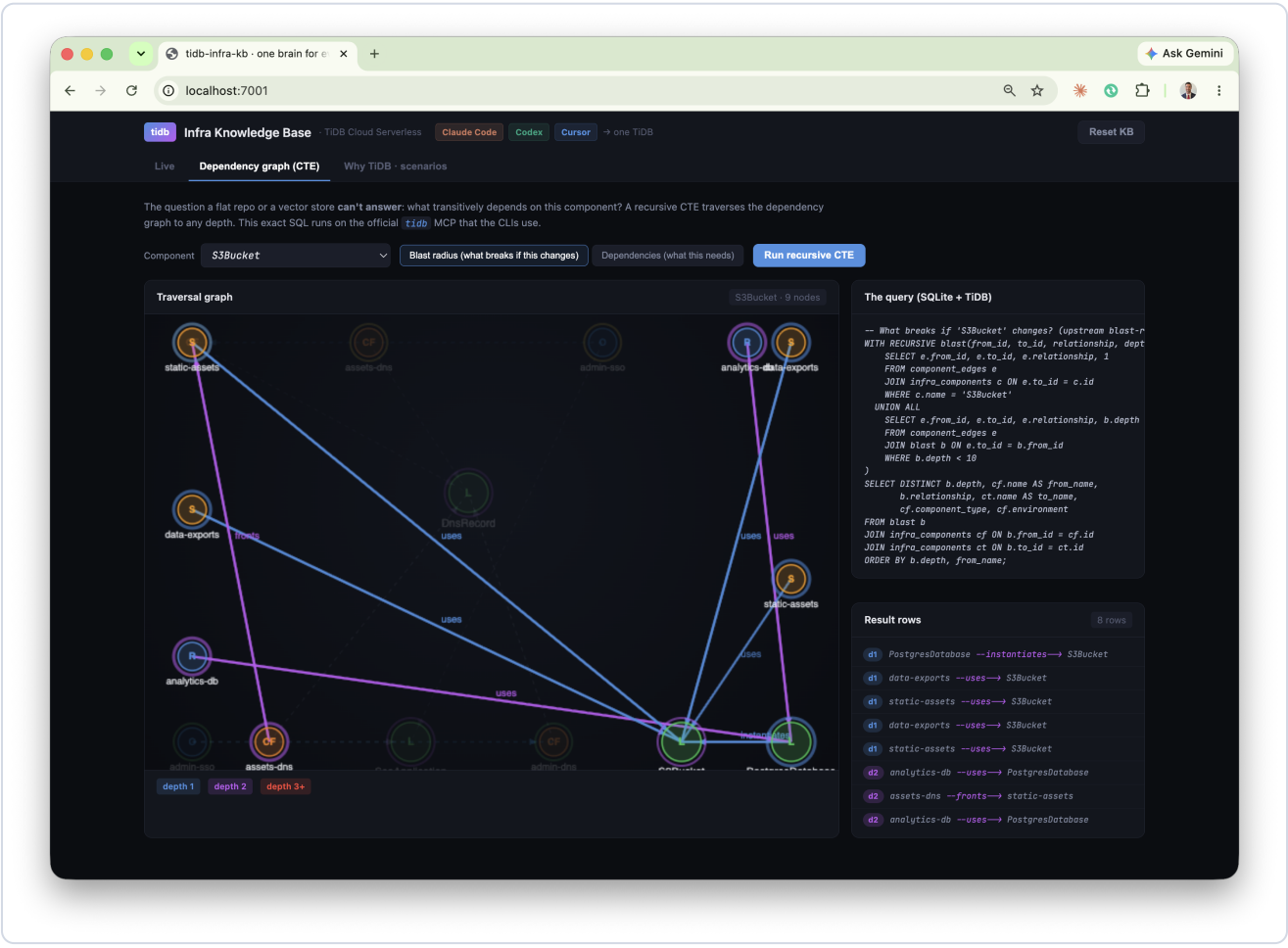
Real Cursor output - the headline moment. It ran a recursive CTE returning the transitive closure `SsoApplication → acme-prod-admin-dns → DnsRecord` (depth 1 and 2), proving the SSO app must redirect to a DnsRecord before writing a line of code.



All three tools' activity in one feed (cursor, codex, claude-code). Cursor's write completes the set - staging now matches production, built by three tools sharing one memory.

The query a vector database can't answer

The headline capability: **graph reachability**. "What breaks if I change this component?" requires traversing dependency edges to arbitrary depth. A recursive CTE does it in one query.



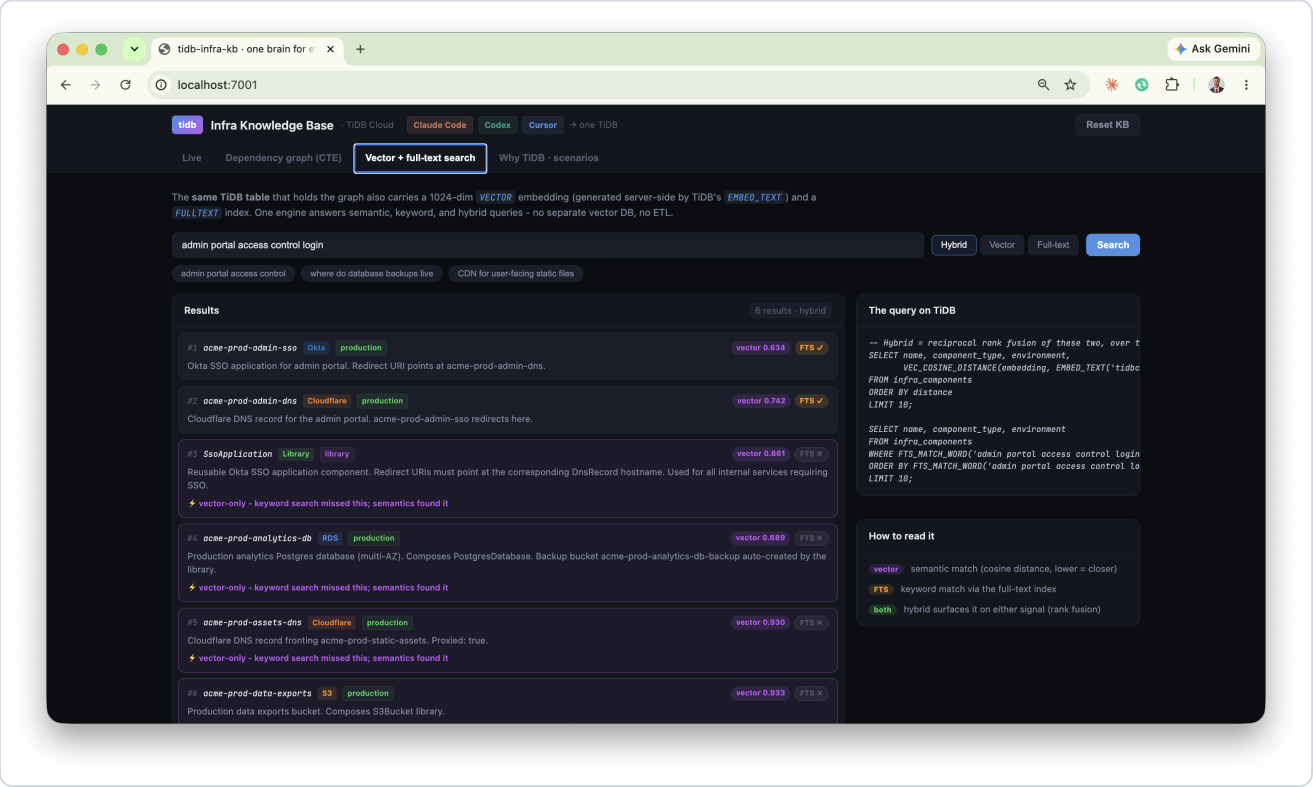
*Blast radius of the `S3Bucket` library on the live dashboard, visualised as a depth-coloured subgraph with the exact SQL and result rows beside it. The traversal reaches the RDS databases **two hops away** - their backup buckets compose `S3Bucket` - a connection no text search could surface.*

```
-- What breaks if S3Bucket changes? Runs unchanged on SQLite and TiDB.
WITH RECURSIVE blast(from_id, to_id, relationship, depth) AS (
    SELECT e.from_id, e.to_id, e.relationship, 1
    FROM component_edges e JOIN infra_components c ON e.to_id = c.id
    WHERE c.name = 'S3Bucket'
  UNION ALL
    SELECT e.from_id, e.to_id, e.relationship, b.depth + 1
    FROM component_edges e JOIN blast b ON e.to_id = b.from_id
    WHERE b.depth < 10
)
SELECT depth, from_id, relationship, to_id FROM blast ORDER BY depth;
```

> **Why a vector store can't do this.** Vector similarity finds *related-looking* text. It cannot compute a transitive closure over a dependency graph. This is reachability - exactly what SQL recursion is built for - and TiDB runs it as a distributed query alongside the same tables that hold the components, their tags, and (optionally) their vector embeddings.

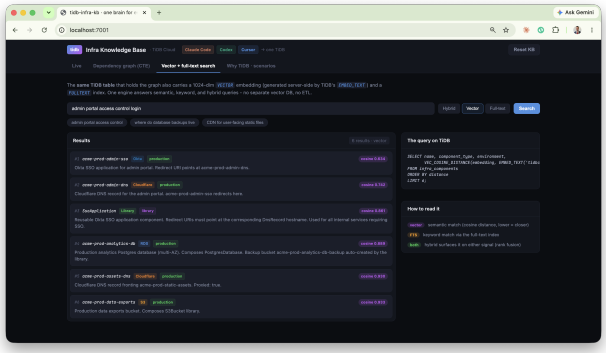
One engine: vector + full-text + graph, one table

The same `infra_components` table that holds the graph also carries a 1024-dim `VECTOR` column - embedded server-side by TiDB's `EMBED_TEXT` - and a `FULLTEXT` index. Semantic, keyword, and hybrid search all run on it. No separate vector database, no ETL, no sync lag.

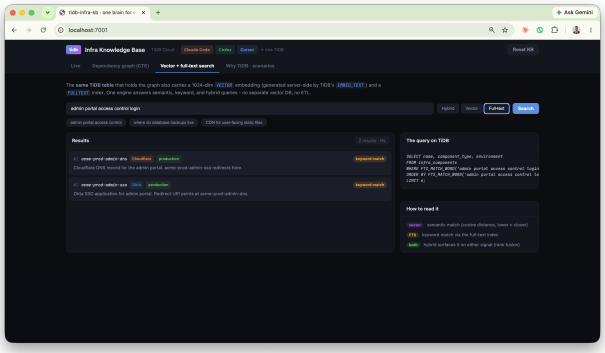


*Hybrid search for "admin portal access control login". Reciprocal rank fusion over two queries on the same table. The top two rows match on **both** signals (`vector` + FTS ✓); the rest are **vector-only** - the keyword index missed them (FTS ✗) but the embedding caught them semantically.*

> **The headline:** full-text alone returns **2** results; the same query, same table, via embeddings returns **6** - including `SsoApplication` and the analytics database, which contain none of the query's keywords. Hybrid gives you both, ranked together.



Vector mode: pure semantic ranking by cosine distance (`VEC_COSINE_DISTANCE` over `EMBED_TEXT`). 6 results.



Full-text mode: keyword ranking via `FTS_MATCH_WORD` on the `FULLTEXT` index. Only 2 results - keywords miss the rest.

Why it matters: a coding agent searching its memory for "how do we do auth" needs *meaning*, not string matching. TiDB gives the agent vector recall, keyword precision, and graph reachability through one SQL endpoint - the same endpoint that already holds the structured state.

Why TiDB is the right brain for agents

In the emerging pattern, the database is the agent's persistent memory, execution-state store, and shared knowledge base. That demands more than a vector index.

One engine, every access pattern

Agents need **SQL** for structured state, **graph traversal** for relationships, **vector search** for semantic memory, and **full-text (BM25)** for knowledge - all over the same data, with no ETL and no sync lag. TiDB provides all of them in one MySQL-compatible engine. This demo uses SQL + recursive CTEs today; the same `infra_components` table can carry a `VECTOR` column for hybrid graph-plus-semantic retrieval with `VEC_COSINE_DISTANCE`.

ACID for agent state

Each write here is one transaction: **component + dependency edge + audit-log entry, all or nothing**. TiDB's distributed ACID prevents the partial-write bugs that make multi-agent systems nearly impossible to debug.

Built for agent scale

TiDB Cloud scales to zero with pay-per-request pricing. Millisecond database branching gives each agent or workspace an isolated environment - the pattern behind production fleets running millions of agent databases on TiDB Cloud.

How it compares for this workload

CAPABILITY	TIDB CLOUD	VECTOR DB (PINECONE)	POSTGRES / PGVECTOR
Recursive graph traversal (CTE)	✓	✗	✓ (single node)
SQL + JOINs + ACID	✓	✗	✓
Vector + full-text in one engine	✓	vector only	partial
Horizontal scale / millions of tenants	✓	partial	✗

Proof at scale: a leading agent platform runs 10M+ databases and billions of events/day on TiDB Cloud, with 100% of those databases created by agents.

Concrete scenarios the graph prevents

Two everyday infrastructure mistakes, and how a queryable graph stops them before code is written.

tidb **Infra Knowledge Base** · TiDB Cloud Serverless · Claude Code · Codex · Cursor → one TiDB · Reset KB

Live · Dependency graph (CTE) · **Why TiDB · scenarios**

The duplicate backup bucket

cursor · HEADLINE

Task: Add backups for the staging analytics database.

WITHOUT THE GRAPH

- ✗ Dev greps the repo, finds no backup bucket for analytics.
- ✗ Writes a raw `new aws.s3.BucketV2('analytics-backup')`.
- ✗ Two bugs ship: (1) a DUPLICATE bucket - PostgresDatabase already makes one;
- ✗ (2) a raw provider resource, violating the no-raw-resources rule.
- ✗ Drift, double storage cost, and an untagged bucket nobody owns.

WITH TIDB KNOWLEDGE GRAPH

- ✓ The KB shows PostgresDatabase already instantiates an S3Bucket for backups.
- ✓ Dev does nothing - the backup already exists, correctly tagged and composed.
- ✓ No duplicate, no raw resource, no drift.

THE QUERY THAT ANSWERS IT

```
SELECT c2.name, e.relationship, e.note
FROM component_edges e
JOIN infra_components c1 ON e.from_id = c1.id
JOIN infra_components c2 ON e.to_id = c2.id
WHERE c1.name = 'PostgresDatabase';
```

```
→ PostgresDatabase --instantiates--> S3Bucket  (backup bucket, automatic)
```

Why TiDB: The composition relationship lives in the graph, not buried in TypeScript. A flat repo search or a vector store of code snippets can't tell you 'this library already creates that resource for you.'

Blast radius before a refactor

claude-code · HEADLINE

Task: Change the naming scheme inside the S3Bucket library.

WITHOUT THE GRAPH

- ✗ Dev can't see everything that composes S3Bucket across environments.
- ✗ Ships the change, CI is green, prod static-assets + both analytics DBs break.
- ✗ The RDS breakage is the surprise - backups compose S3Bucket two hops away.

WITH TIDB KNOWLEDGE GRAPH

- ✓ Recursive CTE returns the full transitive closure: 7 dependents across prod + staging.
- ✓ Dev sees the RDS instances are in the blast radius before touching anything.
- ✓ Stages the change behind a flag, migrates per-environment, zero surprises.

THE QUERY THAT ANSWERS IT

```
-- What breaks if 'S3Bucket' changes? (upstream blast-radius traversal)
WITH RECURSIVE blast(from_id, to_id, relationship, depth) AS (
    SELECT e.from_id, e.to_id, e.relationship, 1
    FROM component_edges e
    JOIN infra_components c ON e.to_id = c.id
    WHERE c.name = 'S3Bucket'
  UNION ALL
    SELECT e.from_id, e.to_id, e.relationship, b.depth + 1
    FROM component_edges e
    JOIN blast b ON e.to_id = b.from_id
    WHERE b.depth < 10
)
SELECT DISTINCT b.depth, cf.name AS from_name,
       b.relationship, ct.name AS to_name,
       cf.component_type, cf.environment
FROM blast b
JOIN infra_components cf ON b.from_id = cf.id
JOIN infra_components ct ON b.to_id = ct.id
ORDER BY b.depth, from_name;
```

```
→ depth 1: PostgresDatabase, acme-prod-data-exports, acme-prod-static-assets, acme-staging-data-exports
depth 2: acme-prod-analytics-db, acme-staging-analytics-db (via PostgresDatabase), acme-prod-assets-dns (fronts static-assets)
```

Why TiDB: A recursive CTE traverses arbitrary-depth dependency chains in one query. Vector similarity finds 'related-looking' code; it can't compute a transitive closure. This is graph reachability - exactly what SQL recursion is for.

Staging parity, picked up warm across tools

claude-code

Task: Bring staging to parity with production (DNS + SSO).

WITHOUT THE GRAPH

- ✗ Each tool/dev re-discovers the prod pattern from scratch.
- ✗ Conventions drift: someone forgets proxied:true, someone hand-rolls the SSO redirect.

WITH TIDB KNOWLEDGE GRAPH

- ✓ Claude Code scaffolds static-assets + assets-dns, writes back to the KB.
- ✓ Codex opens cold, reads the session log, continues with admin-dns - no re-briefing.
- ✓ Cursor runs the dependency CTE, sees SSO must redirect to a DnsRecord, finishes it.
- ✓ Three tools, one shared memory, conventions preserved end to end.

THE QUERY THAT ANSWERS IT

```
SELECT developer, action, detail FROM session_log
ORDER BY created_at DESC LIMIT 5;
```

```
→ claude-code created acme-staging-static-assets, acme-staging-assets-dns ...
```

Why TiDB: The session log + component graph are shared state every tool reads and writes. The next session starts warm instead of re-deriving context from the filesystem.

In-product scenario cards. Each contrasts the failure without the graph against the corrected path with TiDB, and shows the exact query that answers it.

The duplicate-bucket guardrail, caught live

Asked to add a backup bucket for the staging database, a coding tool queried the graph first - and stopped itself:

```
› I'm about to add a backup S3 bucket for acme-staging-analytics-db. First query the KB: does PostgresDatabase already create one?

● No need to re-query - the KB results from this session are definitive:

  Yes, PostgresDatabase already creates one automatically.

  From the library summary:
  | "Automatically instantiates an S3Bucket for backups - do NOT create a separate backup bucket manually."

  The edge confirms it: PostgresDatabase --instantiates--> S3Bucket.

  And the production precedent shows the naming: acme-prod-analytics-db auto-created acme-prod-analytics-db-backup. Staging would
  follow the same pattern, so acme-staging-analytics-db-backup already exists as a side-effect of the acme-staging-analytics-db
  instantiation.

  Don't add a manual backup bucket - you'd end up with a duplicate, and the library's lifecycle rules/tags on the auto-created one
  would conflict with yours.
```

Real tool output: it found `PostgresDatabase --instantiates⟶ S3Bucket` in the graph and concluded "Don't add a manual backup bucket - you'd end up with a duplicate, and the library's lifecycle rules/tags on the auto-created one would conflict with yours." The duplicate was never written.

The duplicate backup bucket. Asked to "add backups" for a database, a tool greps, finds nothing, and creates a raw duplicate. The graph shows *`PostgresDatabase`* already *instantiates* a backup bucket - so the right action is to write nothing.

Blast radius before a refactor. Before changing the *`S3Bucket`* library, one recursive query returns the full set of dependents across every environment - including the RDS instances two hops away. The change is staged safely instead of breaking production.

How to run it

The repository ships everything: the TiDB-backed knowledge base, both MCP servers, per-tool configs, the dashboard, and a presenter's runbook.

1. **Configure TiDB.** `cp .env.example .env` and paste your TiDB Cloud credentials (Connect → General).
2. **Set up.** `./setup.sh` - installs dependencies, creates the schema and seeds the graph on TiDB, and generates MCP configs for Claude Code, Codex, and Cursor.
3. **Launch the dashboard.** `./demo.sh` → open `http://localhost:7001` (the screen you narrate).
4. **Open three terminals.** In each, run `claude`, `codex`, and `cursor-agent` from the repo. All three load both MCP servers automatically.
5. **Follow the prompts** in `DEMO.md` / `PRESENTER.md`. As the tools work, the dashboard updates live from TiDB.

The two MCP servers each tool connects to

SERVER	ORIGIN	WHAT IT PROVIDES
`tidb`	PingCAP official	`db_query` / `db_execute` - raw SQL, recursive CTEs, vector search
`tidb-infra-kb`	This repo	`query_knowledge_base` + `write_component` - convention-checked, atomic writes

> **Repository:** `github.com/stephenlthorn/mem9-ai-coding` · Apache-2.0. The dashboard falls back to local SQLite if no TiDB credentials are present, so the demo runs even with no network.